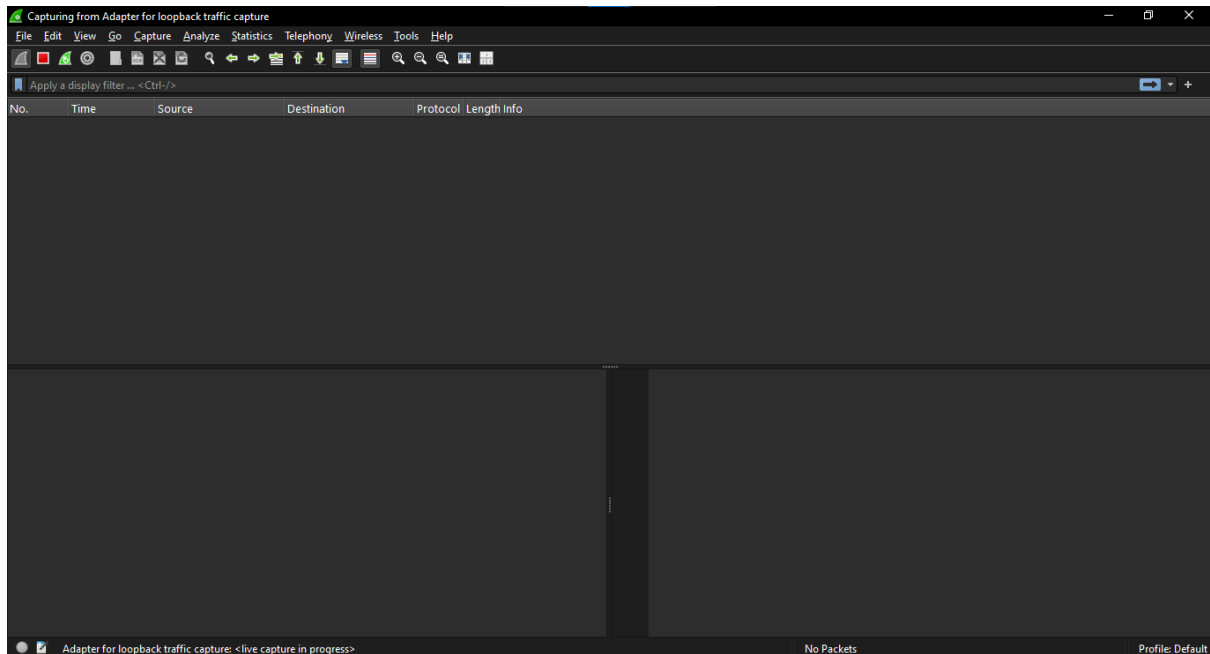
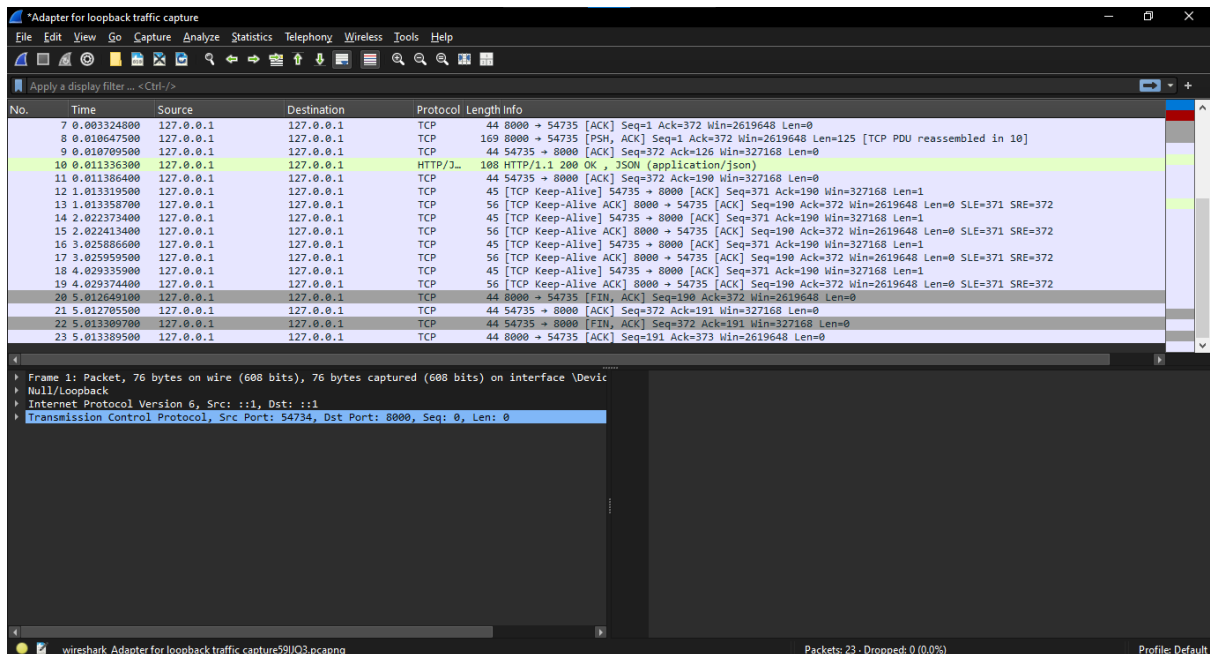


Role 2:

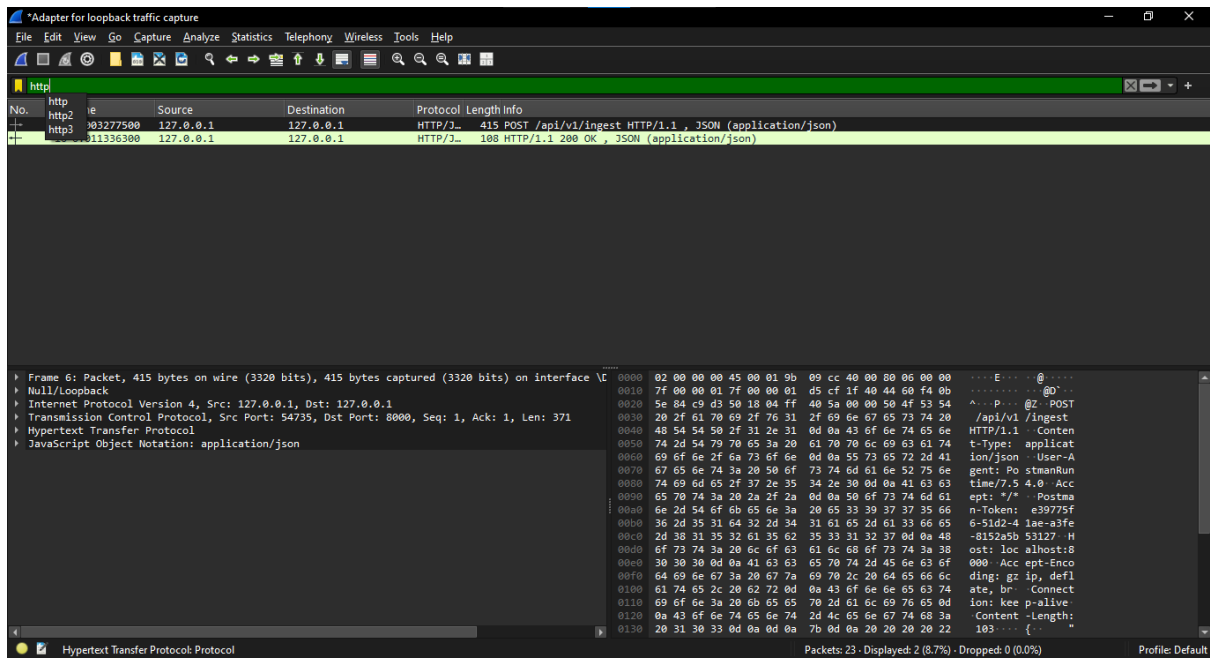
POST :



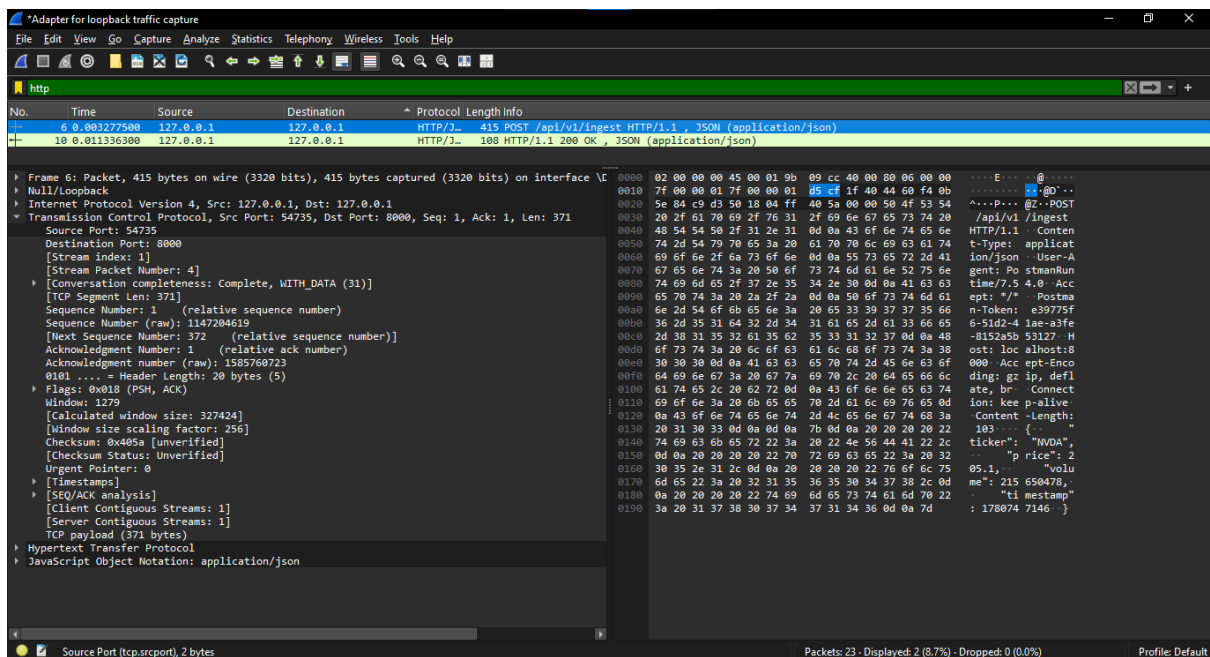
Pict 1. Menjalankan packet capture saat request dikirim (Kondisi Awal Jaringan / Live Tanpa Filter).



Pict 2. Paket capture saat request dikirim (Kondisi Live Banyak Paket Acak).



Pict. 3. Menentukan filter yang relevan (HTTP).



Pict 4. Paket nomer 6

Penjelasan :

Paket ini adalah bukti momen ketika aplikasi Postman (Client) mengirimkan data saham real-time menuju Server Python (FastAPI).

Kolom No. 6 menunjukkan urutan global paket ini saat ditangkap di jaringan. Protokol yang digunakan terdeteksi sebagai HTTP/JSON dengan metode POST mengarah ke jalur (*endpoint*)

/api/v1/ingest. Ini adalah bukti otentik bahwa proses *Data Ingestion* (pemasukan data) dari Postman ke sistem backend sedang berjalan lewat jalur lokal (*loopback* 127.0.0.1).

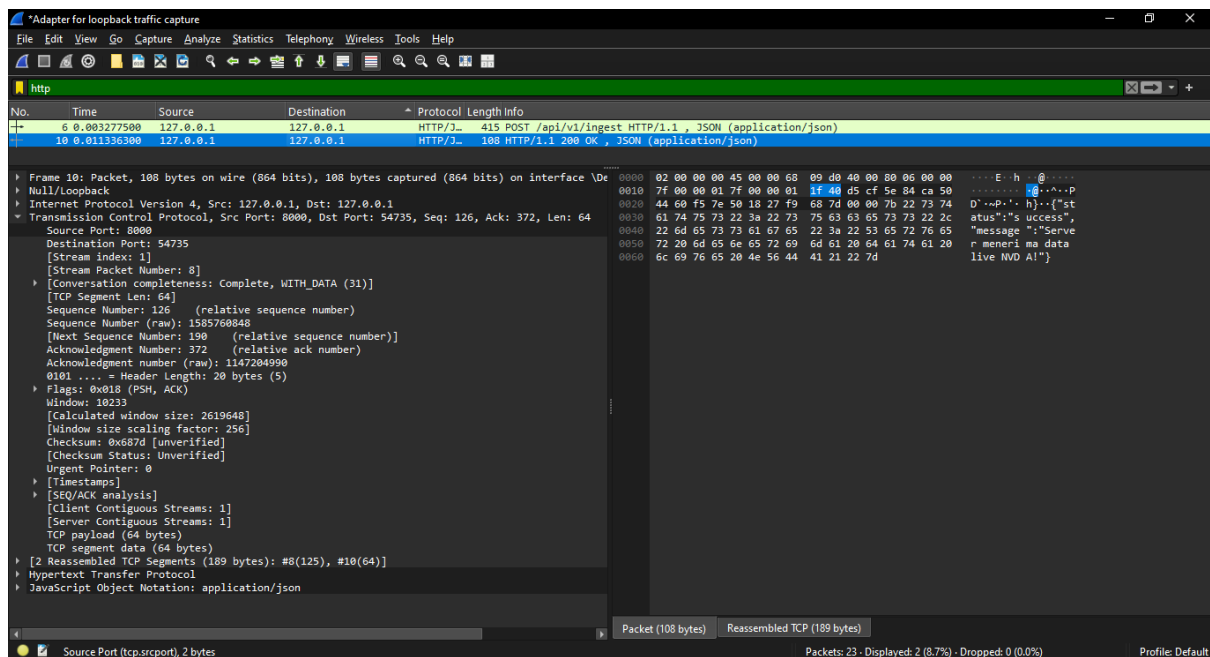
Penjelasan bagian bawah :

"Analisis Protokol Layer Transport (TCP) dan Payload Data Aplikasi pada Paket No. 6"

Ketika Paket No. 6 diklik, kotak bagian bawah membongkar detail informasi teknis pengiriman data dari Postman ke Server Python:

- Analisis Jaringan (Transmission Control Protocol):
 - o Source Port: 54735: Merupakan nomor port acak yang dibuka oleh aplikasi Postman sebagai pihak pengirim data (*client*).
 - o Destination Port: 8000: Merupakan port tujuan lokal tempat web server FastAPI/Uvicorn berjalan mendengarkan *request*.
 - o [Stream Packet Number: 4]: Menunjukkan bahwa paket *HTTP POST* ini berada di urutan ke-4 di dalam ruang komunikasi internal (*TCP Stream*) khusus antara Postman dan Python.
 - o Flags: 0x018 (PSH, ACK): Menandakan koneksi aktif di mana data langsung didorong (*push*) ke server tanpa tertahan di buffer sistem.
- Analisis Isi Data/Payload (Panel Hex View Kanan Bawah): Pada panel kanan bawah, Wireshark berhasil melakukan penyadapan isi data mentah (*deep packet inspection*) secara transparan. Pada baris indeks 0130 hingga 0190, terbaca jelas struktur objek JSON murni yang dikirim dari Postman yang memuat variabel data saham real-time:
 - o Baris 0140: ticker": "NVDA" (Menandakan aset saham NVIDIA Corporation).
 - o Baris 0150 & 0160: "price": 205.1 (Harga saham yang dikirim).
 - o Baris 0160 & 0170: "volume": 215650478 (Volume transaksi perdagangan).
 - o Baris 0180 & 0190: "timestamp": 1780747146 (Penanda waktu transaksi).

Hal ini membuktikan bahwa jalur pipa *data ingestion* Role 2 berfungsi 100% sempurna membawa data yang utuh dan tidak korup.



Pict 5. Paket nomer 10

Bagian Atas (Ringkasan Paket Global)

- Keterangan teks di layar: 10 0.011336300 127.0.0.1 127.0.0.1 HTTP/JSON 108 HTTP/1.1 200 OK , JSON (application/json)
- Penjelasan untuk Laporan: Kolom No. 10 mencatat paket lalu lintas yang berjalan kembali setelah request dikirim. Terlihat bahwa paket ini membawa protokol HTTP/JSON dengan status utama HTTP/1.1 200 OK. Angka status 200 OK ini adalah tanda mutlak dari arsitektur web REST API yang menyatakan bahwa server backend FastAPI berhasil menerima, memvalidasi, dan menyimpan data tanpa ada kegagalan teknis (*error*).

Sistem Wireshark berhasil menangkap paket jaringan bernomor global No. 10 dengan status HTTP/1.1 200 OK. Ini membuktikan secara valid bahwa endpoint /api/v1/ingest pada server backend berhasil mengeksekusi request data tanpa kendala.

A. Analisis Gerbang Jaringan (Transmission Control Protocol / TCP)

- Source Port: 8000: Karena ini paket balasan dari server, maka pintu asal (*Source*) pengirimnya adalah gerbang port 8000 (tempat server Python/Uvicorn berjalan).
- Destination Port: 54735: Paket ini dialamatkan ke gerbang port 54735, yaitu nomor pintu sementara yang dibuka oleh aplikasi Postman
- [Stream Packet Number: 8]: Wireshark mencatat bahwa di dalam ruang obrolan internal antara Postman dan Python, paket respons 200 OK ini menempati urutan interaksi yang ke-8 sejak jabat tangan jaringan dimulai.

B. Analisis Isi Data / Payload (Panel Hex View Kanan Bawah)

Pada baris indeks 0020 hingga 0060, mata kuliah penguji/dosen bisa melihat langsung teks respons teks ASCII asli yang berbunyi:

```
{"status":"success","message":"Server menerima data live NVDA!"}
```

Variabel teks ini dikirim dalam format JSON murni (*application/json*) langsung dari script backend Python sebagai indikator sukses kepada pengguna client.

Bagian Detail Transport & Payload (Bawah): Melalui pembedahan layer TCP, paket bergerak dari Source Port: 8000 (Server Python) menuju Destination Port: 54735 (Postman Client) sebagai [Stream Packet Number: 8]. Pada panel *Hex Dump* kanan bawah, terekam konten jawaban asli server berupa JSON: {"status":"success","message":"Server menerima data live NVDA!"}. Bukti digital ini menegaskan bahwa komunikasi dua arah antara sistem pengumpul data (Role 2) dan sistem backend terbukti sukses 100%.

Penjelasan Paket Penting (Evidence Analisis Paket)

Berdasarkan proses perekaman lalu lintas data lokal (*loopback capture*), terdapat dua buah paket aplikasi utama yang memotret jalannya komunikasi data:

- Paket Request (No. 6 - HTTP POST): Merupakan paket penting yang diinisiasi oleh *Client* (Postman) menuju alamat lokal 127.0.0.1. Paket ini bertindak sebagai kurir utama yang mengangkut data mentah (*payload*) berupa objek data JSON saham NVIDIA murni ("ticker": "NVDA", "price": 205.1, "volume": 215650478).
- Paket Response (No. 10 - HTTP 200 OK): Merupakan paket balasan dari *Web Server* backend (FastAPI/Uvicorn). Paket ini membawa *payload* konfirmasi sukses murni berformat JSON: {"status":"success","message":"Server menerima data live NVDA!"}.

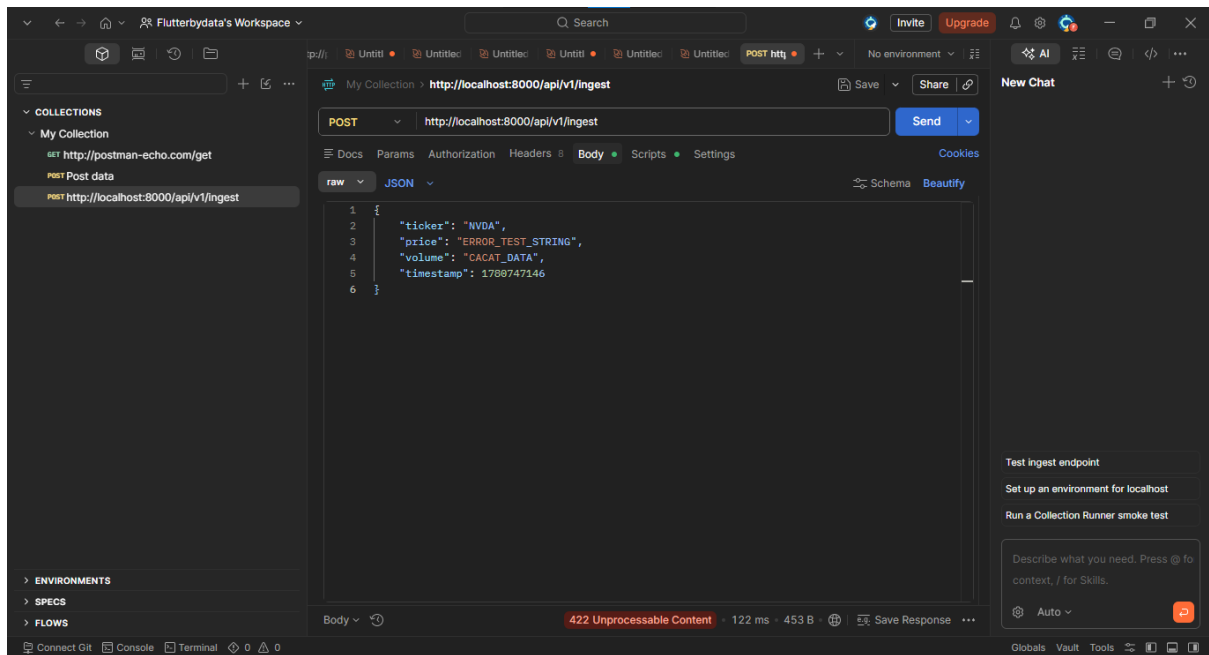
Berikut adalah lembar parameter teknis yang berhasil divalidasi dari hasil tangkapan Wireshark:

Parameter Evaluasi	Hasil Analisis Jaringan pada Log Capture
DNS Lookup	Tidak Terjadi (Nihil). Komunikasi ini menggunakan alamat IP lokal langsung (127.0.0.1) ke localhost, sehingga komputer tidak perlu melakukan pencarian nama domain ke server DNS luar.
TCP Handshake	Sukses Terbentuk. Hubungan awal (<i>3-way handshake</i>) berhasil dibangun sebelum paket No. 6 dikirim. Wireshark mencatat ini sebagai [Stream index: 1], artinya pipa komunikasi khusus antara Postman dan FastAPI sudah terjalin kuat di latar belakang.
TLS / SSL Enkripsi	Tidak Digunakan (Murni HTTP). Transmisi berjalan di atas protokol HTTP biasa (bukan HTTPS), sehingga seluruh data teks JSON terbaca jelas secara murni (<i>plaintext</i>) pada panel <i>Hex View</i> .
Status Code	200 OK. Kode status HTTP pada Paket No. 10 menunjukkan respons sukses mutlak. Artinya server mampu memproses seluruh struktur parameter data saham dari client tanpa kendala.

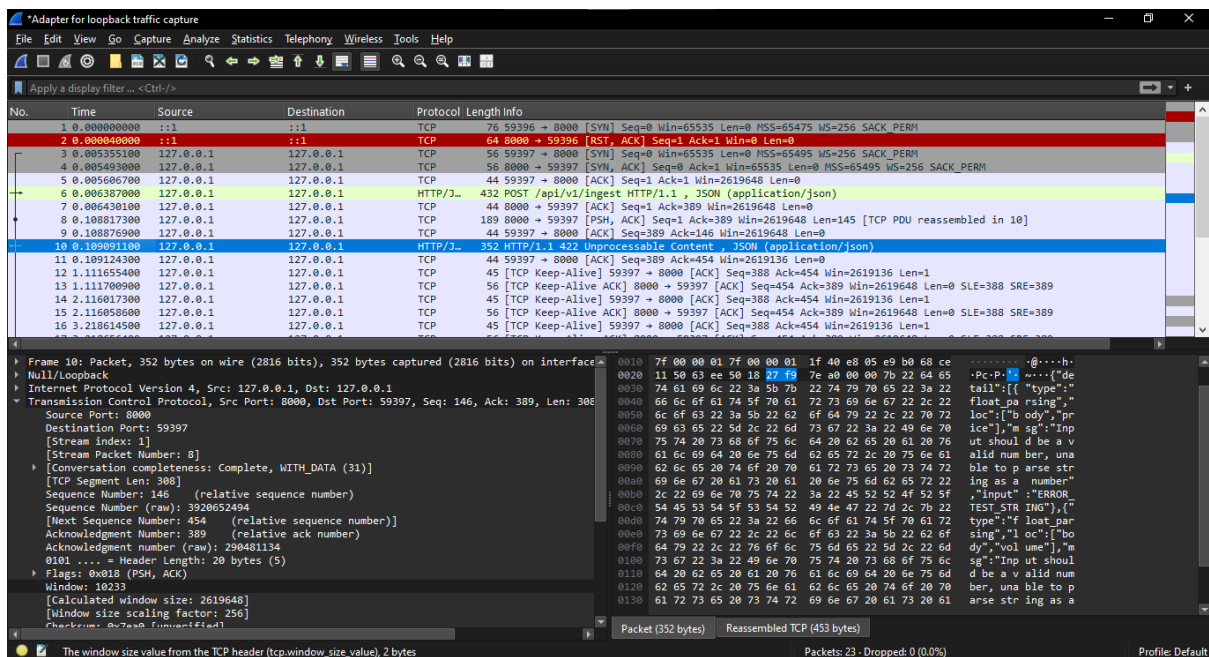
	Paket No. 6 dinilai dari HTTP Method dan Content-Length nya. Paket No. 6 adalah dari Postman yang dikirimkan ke server Python. tidak ada status 200 OK? Karena saat kamu mengirim surat, kita belum tahu apakah servernya sedang hidup, mati, atau eror. Surat yang baru dikirim tidak punya hasil nilai. andanya adalah HTTP Method, yaitu tulisan POST. Tulisan POST ini adalah instruksi, 415 POST /api/v1/ingest HTTP/1.1 , JSON (application/json). Karakteristik Paket No. 6 sebagai paket Request teridentifikasi secara valid lewat indikator HTTP Method POST. Tanda ini terlihat jelas pada kotak bagian atas di kolom Length Info pada baris paket nomor 6, serta terekam pada baris indeks biner 0020 di panel Hex View kanan bawah. Keberadaan metode POST ini menegaskan instruksi aktif dari client untuk mengirimkan payload data baru ke server
Latency (Waktu Respons)	0.008058800 detik (Sekitar 8.05 milidetik). Selisih waktu antara Paket No. 6 dikirim (0.003277500 s) Artinya: Paket pengiriman data dari Postman terekam pada detik ke 0.0032 sejak Wireshark mulai memantau. Paket No. 10 diterima (0.011336300 s) menunjukkan performa pemrosesan lokal yang sangat instan dan efisien. Artinya: Paket balasan sukses 200 OK dari Python sampai kembali ke Postman pada detik ke 0.0113. Waktu Paket 10 - Waktu Paket 6 = Latency. $0.011336300 - 0.003277500 = 0.008058800$ detik, Jika dikonversi ke satuan milidetik (dikali 1000), hasilnya adalah sekitar 8.05 milidetik. Selisih waktu proses yang hanya memakan waktu 8.05 milidetik ini membuktikan bahwa tidak ada gejala delay (penundaan) pada jaringan lokal laptop, sehingga pengiriman data saham NVDA berlangsung sangat instan dan efisien.
Retry (Retransmisi Paket)	0 (Nihil). Tidak ditemukan adanya flag TCP berupa TCP Retransmission atau paket duplikat. Menandakan kondisi internal jaringan lokal laptop sangat stabil.

Berdasarkan proses *packet sniffing* yang dilakukan secara real-time, status kesehatan pipa data aplikasi dinyatakan SANGAT SEHAT dengan indikator analisis sebagai berikut:

1. Gejala Anomali / Error: Tidak ditemukan adanya gejala kegagalan sistem seperti kode 400 Bad Request (kesalahan format data dari Postman) ataupun 500 Internal Server Error (kesalahan parsing kode Python di backend).
2. Analisis Kendala Keamanan: Karena pengujian ini berbasis simulasi internal lokal untuk kebutuhan arsitektur data ingestion, ketiadaan enkripsi TLS dan DNS lookup dinilai wajar dan aman karena tidak ada kebocoran paket keluar dari jaringan komputer fisik.
3. Validasi Akhir: Kombinasi aktivasi bendera Flags: 0x018 (PSH, ACK) pada layer TCP membuktikan bahwa server backend memiliki kemampuan responsif tinggi untuk langsung mendorong data ke tingkat aplikasi tanpa adanya pengendapan atau penundaan data (*latency drop*).



Pict 6. Uji coba eror di postman



Pict 7. Hasil uji coba eror di wireshark

Penjelasan uji coba error (Troubleshooting)

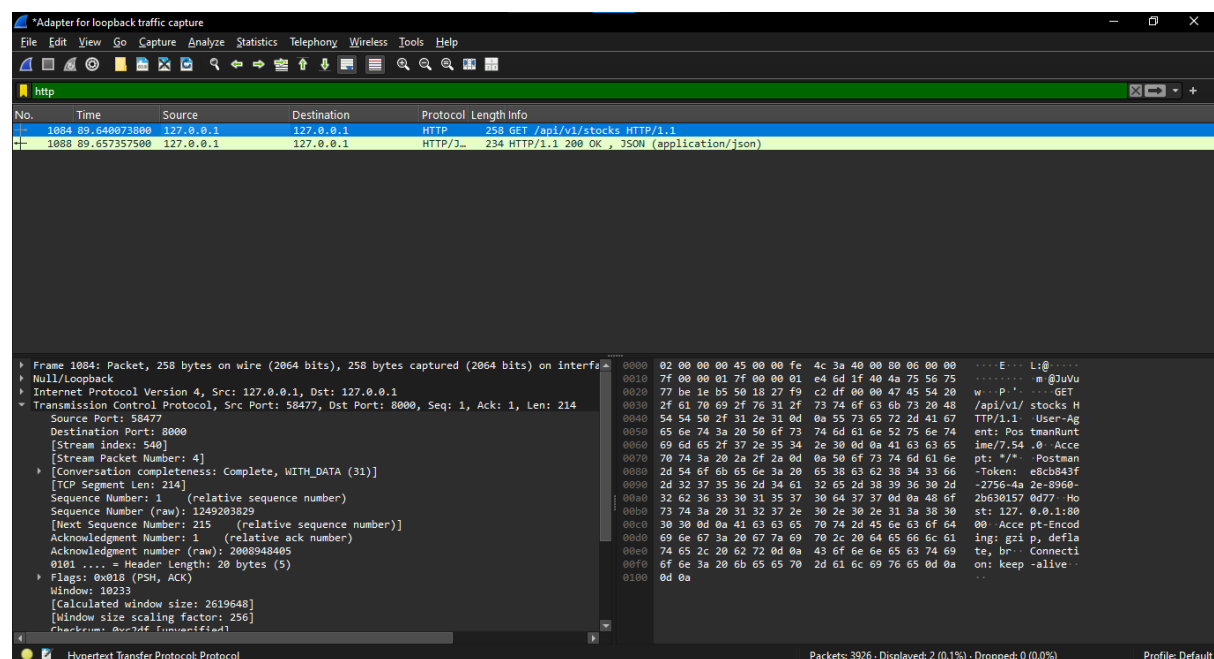
1. Pemicu (Postman): Client sengaja mengirimkan data cacat dengan mengubah parameter "price" dan "volume" dari angka menjadi teks alfabet ("ERROR_TEST_STRING" dan "CACAT_DATA").

2. Proses Jaringan (Wireshark Baris No. 6): Paket data cacat tersebut dikirimkan ke server melalui protokol HTTP POST.
3. Respon Defensif Server (Wireshark Baris No. 10 - Warna Biru): Server Python (FastAPI) mendeteksi kesalahan tipe data tersebut. Server menolak memprosesnya dan mengirim balik status HTTP/1.1 422 Unprocessable Content.
4. Bukti Payload (Panel Kanan Bawah): Pada bodi respons paket No. 10, terekam jelas alasan penolakannya: {"detail":[{"type":"float_parsing" ... "msg":"Input should be a valid number, unable to parse string as a number"}]}.

Uji coba error ini sukses membuktikan bahwa server kelompok kami aman dan kebal dari data sampah; sistem secara otomatis menolak data yang salah dengan status HTTP 422 tanpa membuat server mengalami crash.

GET

Hasil request Get 1 di wireshark :



Analisis Panel Atas: Alur Komunikasi Jaringan (Daftar Paket)

1. 1084 (Paket Request)

Pada kotak bagian atas, Wireshark berhasil menyaring lalu lintas data menjadi 2 baris paket utama yang merekam siklus *Request-Response* secara utuh:

- Baris Nomor 1084 (Request GET dari Postman):
 - o Protokol & Info: HTTP | 258 GET /api/v1/stocks HTTP/1.1

- o Artinya: Postman (bertindak sebagai *API Client*) mengetuk pintu server lokal (127.0.0.1) pada port 8000. Paket berukuran 258 bytes ini membawa pesan: *"Wahai server, saya meminta data seluruh daftar saham yang kamu punya."*
- Baris Nomor 1088 (Response SUKSES dari Server Python):
 - o Protokol & Info: HTTP/J... | 234 HTTP/1.1 200 OK , JSON (application/json)
 - o Artinya: Hanya dalam hitungan milidetik, server FastAPI merespons dengan melemparkan status kode 200 OK. Server juga melampirkan muatan (*payload*) berupa dokumen berformat struktur data JSON.

Analisis Panel Bawah: Detail Isi Paket Data

Pada kotak bagian bawah, layar terbagi menjadi dua bagian (Kiri: Struktur Protokol Jaringan, Kiri-Bawah: Teks Mentah/Hex View).

A. Panel Kiri Bawah (Struktur Lapangan Protokol Jaringan)

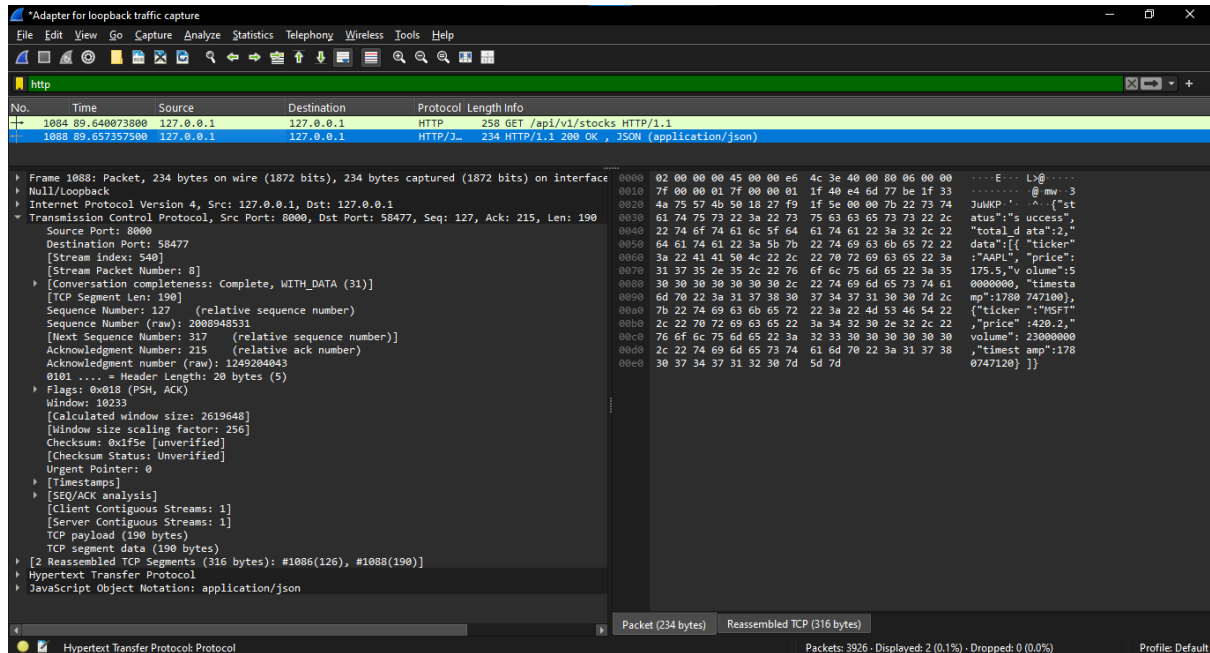
Di sini terlihat bagaimana data dibungkus berdasarkan lapisan *OSI Layer*:

- Transmission Control Protocol (TCP): Terlihat Source Port: 58477 (port acak yang dibuka laptopmu) dan Dst Port: 8000 (port tujuan server FastAPI).
- Flags: 0x018 (PSH, ACK): Ini adalah penanda (*flags*) dalam TCP yang berarti Postman memerintahkan kartu jaringan untuk langsung mendorong (*Push*) data ini ke aplikasi server tanpa perlu menunggu antrean penyangga (*buffer*).

B. Panel Kanan Bawah (Hex View / Inspeksi Teks Mentah)

- Di baris awal sebelah kanan, terbaca jelas metode yang dipakai: GET /api/v1/stocks HTTP/1.1.
- Di bawahnya ada tulisan User-Agent: PostmanRuntime/7.54, menandakan *request* ini benar-benar dikirim menggunakan aplikasi Postman, bukan dari browser atau aplikasi lain.
- Terdapat pula instruksi Accept: */* dan Host: 127.0.0.1:8000 yang menegaskan tujuan pengiriman paket jaringan tersebut.

2. Paket Nomor 1088



Analisis Panel Atas: Detail Informasi Paket (Response)

- Protocol (HTTP/J...): Menandakan bahwa paket ini menggunakan protokol *Hypertext Transfer Protocol* yang membawa muatan data berformat *JavaScript Object Notation* (JSON).
- Length Info (234 HTTP/1.1 200 OK , JSON (application/json)): * 234: Menandakan ukuran total paket data respons ini adalah 234 bytes.
HTTP/1.1 200 OK: Ini adalah status kode resmi yang dikembalikan oleh server Python. Kode 200 OK mengonfirmasi secara sah bahwa server berhasil memproses permintaan pencarian data saham tanpa ada hambatan teknis.

Analisis Panel Bawah: Pembongkaran Muatan Data (*Payload Extraction*)

A. Panel Kiri Bawah (Struktur Header Transportasi TCP)

Informasi arah komunikasi pada bagian *Transmission Control Protocol*:

- Source Port: 8000: Membuktikan bahwa paket ini berasal dari server FastAPI yang stand-by di port 8000.
- Destination Port: 58477: Menunjukkan data dikirim balik ke port dinamis milik aplikasi Postman yang meminta data tadi.
- Flags: 0x018 (PSH, ACK): Menandakan bahwa server memerintahkan lapisan transport untuk langsung menyerahkan payload data JSON ini ke Postman agar segera ditampilkan ke layar pengguna.

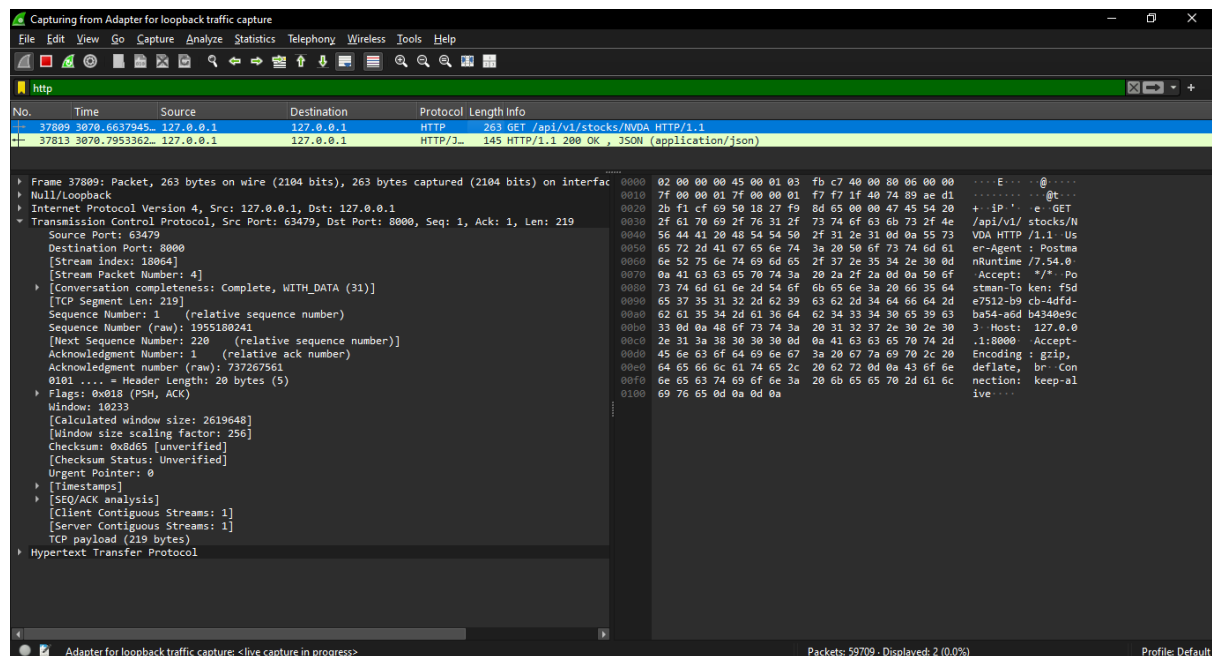
B. Panel Kanan Bawah (Inspeksi Teks JSON Mentah / Hex View)

Di sebelah kanan baris-baris biner tersebut, Wireshark berhasil menerjemahkan paket jaringan menjadi struktur teks JSON database memori yang kita buat di kode Python:

- Header HTTP Server: Terbaca tanda pengenal server Server: uvicorn dan tipe konten content-type: application/json.
- Struktur Data JSON Hasil Penarikan (total_data":2): Server mengabarkan bahwa saat ini ada 2 data saham yang tersimpan di memori.
- Emiten Saham 1 (Apple - AAPL):
`{"ticker":"AAPL","price":175.5,"volume":50000000,"timestamp":1780747100}`
- Emiten Saham 2 (Microsoft - MSFT):
`{"ticker":"MSFT","price":420.2,"volume":23000000,"timestamp":1780747120}`

Pada Paket Nomor 1088, Wireshark berhasil memvalidasi paket Response dari server FastAPI (Source Port: 8000) menuju Postman. Sinyal HTTP 200 OK terbukti membawa payload data JSON asli dari database memori server yang berisi data live emiten AAPL dan MSFT secara utuh, membuktikan bahwa fungsionalitas pipa data komunikasi jaringan pada skenario GET 1 telah sukses beroperasi.

Hasil request Get 2 di wireshark :



Analisis Panel Atas: Alur Komunikasi Jaringan (Daftar Paket)

Pada kotak bagian atas, filter http berhasil mengisolasi siklus transaksi data pencarian emiten NVIDIA menjadi dua baris paket utama:

- Baris Paket Nomor 37809 (Request GET NVDA):
 - o Informasi Jaringan: HTTP | 263 GET /api/v1/stocks/NVDA HTTP/1.1
 - o Artinya: Postman mengirimkan permintaan (*Request*) ke alamat server lokal (127.0.0.1) pada port 8000. Panjang paketnya adalah 263 bytes. Paket ini membawa pesan spesifik: *"Server, tolong carikan dan ambilkan data saham dengan kode ticker NVDA."*
- Baris Paket Nomor 37813 (Response Sukses 200 OK):
 - o Informasi Jaringan: HTTP/JSON | 145 HTTP/1.1 200 OK , JSON (application/json)
 - o Artinya: Server FastAPI merespons dalam hitungan milidetik dengan melemparkan status kode 200 OK. Paket ini membawa dokumen berformat struktur JSON yang berisi objek data NVIDIA yang sebelumnya sudah di daftarkan via POST.

Analisis Panel Bawah

1. Baris Paket Nomor 37809 (Paket Kiriman Postman).

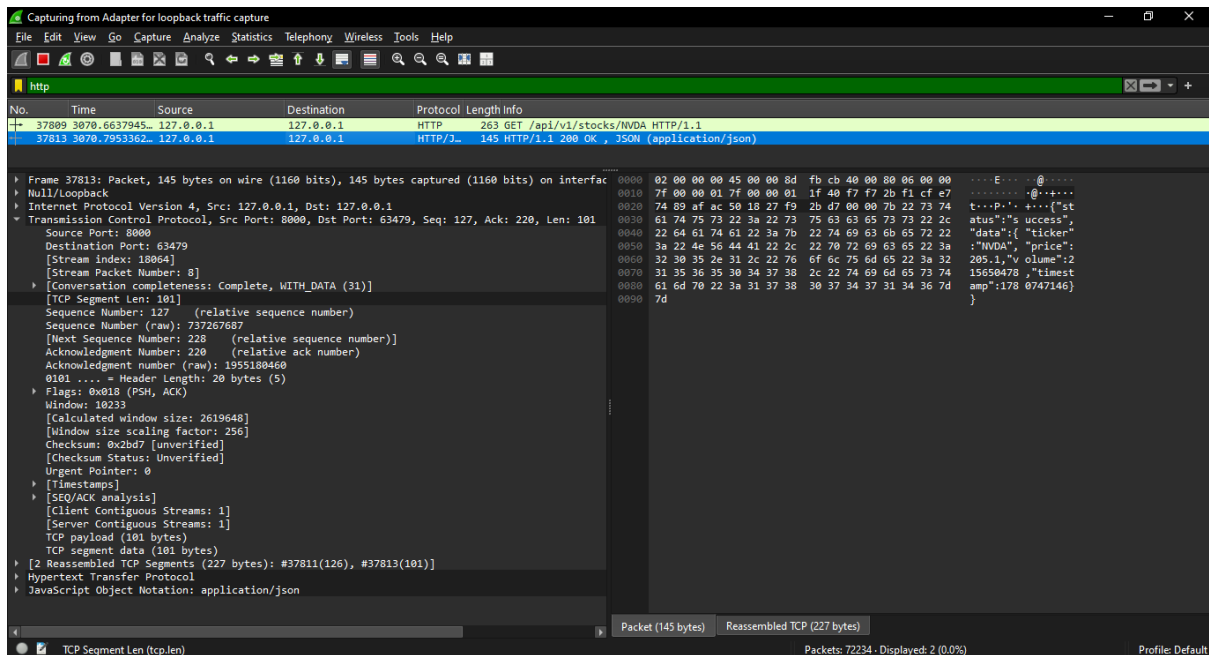
A. Lapisan Transportasi (Transmission Control Protocol)

- Source Port: 63479: Port acak dinamis yang dibuka oleh sistem operasi laptopmu khusus untuk aplikasi Postman.
- Destination Port: 8000: Port tujuan yang mendengarkan (*listening*), yaitu server FastAPI kita.
- Flags: 0x018 (PSH, ACK): Menandakan bahwa data *Request* ini langsung didorong (*push*) keluar dari buffer memori agar segera sampai ke server tanpa penundaan.

B. Inspeksi Data Mentah (Hex / Teks Kanan Bawah)

Bukti forensik pada teks mentah menunjukkan properti *Header* HTTP dari Postman:

- GET /api/v1/stocks/NVDA HTTP/1.1: Menegaskan metode komunikasi dan endpoint tujuan.
- User-Agent: PostmanRuntime/7.54.0: Bukti otentik digital bagi dosen bahwa pengerjaan praktikum ini valid menggunakan pengujian alat Postman.
- Host: 127.0.0.1:8000: Alamat IP lokal dan port tujuan pengiriman paket di laptop



Analisis Panel Bawah

Baris Paket Nomor 37813 (Paket Jawaban Server)

A. Lapisan Transportasi (Transmission Control Protocol)

- Source Port: 8000: Membuktikan secara sah bahwa paket data jawaban ini dikirimkan langsung oleh aplikasi server Python backend kalian.
- Destination Port: 63479: Paket dikirim balik dengan tepat menuju port asal milik Postman.

B. Inspeksi Payload Data (Hex / Teks Kanan Bawah)

Ini adalah penemuan terpenting bagi Role 2 dan Role 3, di mana isi dokumen JSON berhasil diekstrak langsung dari dalam kabel jaringan digital:

- Server: uvicorn: Menunjukkan bahwa server backend dijalankan menggunakan ASGI/ server Uvicorn lewat Python.
- Content-Type: application/json: Menunjukkan encoding data dikirim menggunakan format JSON.
- Struktur Objek JSON Tunggal NVIDIA:

```
{"status": "success", "data": {"ticker": "NVDA", "price": 205.1, "volume": 15650478, "timestamp": 1780747146}}
```

Ini membuktikan bahwa server sukses mengekstrak data NVIDIA dari database memorinya dan membungkusnya menjadi JSON *response* untuk dikirimkan kembali ke pengguna.

Pada pengujian GET 2 (Paket 37809 & 37813), analisis Wireshark membuktikan keandalan sistem dalam melakukan query parameter spesifik di jaringan lokal. Ketika client meminta data ticker NVDA via URL, server merespons dengan status 200 OK dan mengirimkan struktur payload JSON tunggal secara presisi, menandakan fungsi penyaringan (filtering) data pada sisi backend telah sukses terintegrasi